AI 201 PA 3 Report
Implementing the Backpropagation Algorithm

Adriane Jone A. Abunda (2025-22450)

Date of Submission: March 27, 2026

1. INTRODUCTION

Artificial neural networks, particularly the Multilayer Perceptron (MLP), remain foundational architectures for supervised classification owing to their universal approximation capability and principled gradient-based learning [1]. When coupled with the Backpropagation algorithm, an MLP can learn complex, nonlinear decision boundaries by iteratively adjusting its weights to minimize a loss function over labeled training data [2].

A persistent challenge in practical classification tasks is *class imbalance*: when the number of instances per class varies dramatically, a naïve network can achieve deceptively high accuracy by predicting only the majority class. The Synthetic Minority Over-sampling Technique (SMOTE) [3] addresses this by generating synthetic instances along line segments connecting minority-class neighbours, thereby enriching the training distribution without simple duplication.

This report documents the implementation of a 2-layer MLP trained with mini-batch Stochastic Gradient Descent (SGD) and momentum on an 8-class dataset comprising 354 features. Two network variants are compared: Network A, which employs the *Tanh* activation function in both hidden layers, and Network B, which substitutes *Leaky ReLU*. Both networks share the same architecture and output-layer activation (Logistic). The report covers dataset partitioning and SMOTE balancing, hyperparameter tuning, learning-curve analysis, timing comparisons, and a comprehensive evaluation using the confusion matrix, Accuracy, Precision, Recall, F1 Score, and the Matthews Correlation Coefficient (MCC).

2. OBJECTIVES

The objectives of this programming assignment are as follows. First, to implement a fully functional multilayer perceptron from scratch using only NumPy, including forward propagation, backpropagation, and momentum-based mini-batch stochastic gradient descent. Second, to apply SMOTE to address the severe class imbalance in the training data, producing a balanced dataset for training and validation. Third, to tune hyperparameters (hidden layer sizes, learning rate, and batch size) systematically using 30-epoch experiments and select configurations that minimize validation SSE. Fourth, to train and evaluate two network architectures, Network A (Tanh-Tanh-Logistic) and Network B (LeakyReLU-LeakyReLU-Logistic), for 200 epochs and compare their performance on accuracy, F1 score, MCC, and training time. Fifth, to generate test set predictions using the trained network weights.

3. METHODOLOGY

3.1 Dataset Partitioning

The dataset contains 3,486 instances, each described by 354 numerical features, with 8 class labels. The class distribution is severely imbalanced, as summarised in Table 1.

Table 1: Original class distribution.

Class	Count	% of Total
1	1,625	46.6%
2	233	6.7%
3	30	0.9%
4	483	13.9%
5	287	8.2%
6	310	8.9%
7	52	1.5%
8	466	13.4%

Class 3 has only 30 instances while Class 1 has 1,625, yielding a 54:1 imbalance ratio. Training a neural network directly on this distribution would bias the model toward the majority class and result in poor recall for minority classes such as Classes 3 and 7.

3.2 SMOTE Balancing

To address this imbalance, the Synthetic Minority Over-sampling Technique (SMOTE) [3] was applied using the `imblearn` library. SMOTE works by selecting a minority-class sample, identifying its k-nearest neighbours within the same class, and generating a synthetic instance at a random point along the line segment connecting the sample to one of its neighbours. This process is repeated until the minority class matches the majority class in size. Importantly, SMOTE creates novel data points rather than duplicating existing ones, which provides richer gradient information during training and reduces the risk of overfitting to repeated instances.

After SMOTE, all 8 classes were equalized to 1,625 samples each, producing a balanced dataset of 13,000 samples. The balanced dataset was then randomly shuffled and the labels were one-hot encoded into 8-dimensional binary vectors for compatibility with the MLP's output layer.

3.3 Train/Validation Split

The balanced data was then shuffled and one-hot encoded into 8-dimensional target vectors. The first 800 samples were reserved as the validation set, and the remaining 12,200 samples formed the training set.

Because the data was shuffled from a perfectly balanced distribution, taking the first 800 samples yields approximately 100 samples per class in the validation set (actual counts: 112, 108, 97, 88, 116, 92, 89, 98 for Classes 1–8), which is close to uniform as required by the assignment. The `imblearn` library was used exclusively for SMOTE data generation; all neural network computations were implemented using only NumPy.

3.4 Network Architecture

Both networks share the same architecture: a 354-input layer, two hidden layers of sizes [100, 50], and an 8-neuron output layer using the Logistic activation function with slope parameter $a = 2.0$. Weights were initialized using Haykin's heuristic: uniformly sampled from $[-2.4/\text{fan_in}, 2.4/\text{fan_in}]$, where fan_in is the number of inputs to a layer.

Network A uses Tanh activations in both hidden layers, with parameters $a = 1.716$ and $b = 2/3$. Network B uses Leaky ReLU activations in both hidden layers with a slope of 0.01 for negative inputs.

3.5 Training Procedure

Training used mini-batch stochastic gradient descent with momentum (the Generalized Delta Rule). The loss function is the Sum of Squared Errors (SSE). The update rule for each weight matrix $\mathbf{W}$ is:

$$\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_j(n) \tag{1}$$

where η is the learning rate and $\alpha = 0.9$ is the momentum constant.

3.6 Hyperparameter Tuning

Before committing to 200-epoch full training, systematic 30-epoch experiments were conducted. For Network A, the search space covered hidden layer sizes [64, 32], [100, 50], [128, 64], and [200, 100]; learning rates of 0.01 and 0.001; and batch sizes of 8, 16, and 32, totaling 18 configurations. The configuration with the lowest validation SSE was selected.

Table 2: Network A hyperparameter search space (momentum $\alpha = 0.9$ fixed).

Hidden Layers	Learning Rate (η)	Batch Sizes
$[64, 32]$	0.01, 0.001	8, 16, 32
$[100, 50]$	0.01, 0.001	8, 16, 32
$[128, 64]$	0.001	8, 16, 32
$[200, 100]$	0.001	8, 16, 32

The best configuration was $[100, 50]$ hidden nodes, $\eta = 0.001$, batch size = 16, achieving a validation SSE of 19.75 and 18 misclassifications after 30 epochs. Notably, the $[64, 32]$ network with $\eta = 0.001$ and batch size 16 achieved a competitive SSE of 22.30, while the much larger $[200, 100]$ network did not improve upon the medium-sized architecture, suggesting that the additional capacity was unnecessary for this problem and may have slowed convergence due to the higher parameter count. The $\eta = 0.01$ learning rate consistently produced higher validation SSE across all architectures, indicating that this rate was too aggressive for the momentum-based updates.

For Network B, hidden layer sizes were fixed to [100, 50] (from Network A's best result), and only learning rate and batch size were varied over 6 configurations. Momentum $\alpha = 0.9$ was fixed for all experiments.

Table 3: Network B hyperparameter search space (Hidden Layers and momentum fixed).

Learning Rate (η)	Batch Sizes
0.01	8, 16, 32
0.001	8, 16, 32

The best configuration was again $\eta = 0.001$, batch size = 16, achieving a validation SSE of 20.04 and 20 misclassifications after 30 epochs. The consistency of optimal hyperparameters across both activation functions suggests that the learning dynamics for this dataset and architecture are relatively stable with respect to the hidden-layer activation choice.

4. EXPERIMENTAL RESULTS

Both networks were trained for 200 epochs using the selected hyperparameters.Table 4 summarizes the final training configuration.

Table 4: Final training configuration for both networks.

Parameter	Network A	Network B
Hidden Nodes	$100 \to 50$	$100 \to 50$
Hidden act.	a=1.716, b=2/3	slope = 0.01
Output act.	a=2.0	a=2.0
Learning rate (η)	0.001	0.001
Momentum (α)	0.9	0.9
Batch size	16	16
Epochs	200	200

4.1 Learning Curves

4.1.1 Network A

Figure 1 presents the learning curves for Network A. Training SSE decreases steadily from approximately 5,496 at epoch 1 to approximately 56 at epoch 200. Validation SSE, measured every 5 epochs, follows a corresponding decline from approximately 350 to 7.13, with occasional spikes at epochs 30, 60, and 150 that quickly recover. These transient spikes are characteristic of momentum-based mini-batch SGD: with $\alpha = 0.9$, the accumulated velocity from previous updates can temporarily overshoot when particular batch orderings produce noisy gradient estimates. The rapid recovery confirms these are not indicative of persistent overfitting.

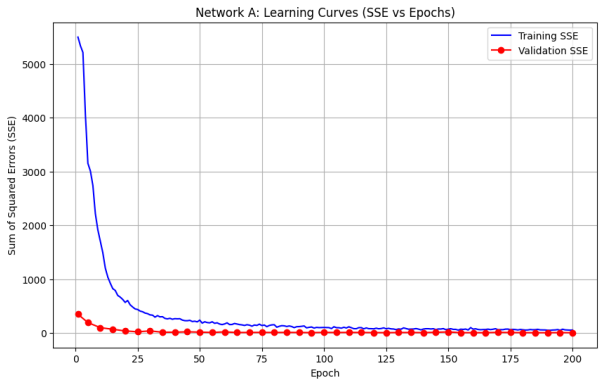

Figure 1: Network A (Tanh): Training and validation SSE versus epoch number. Validation SSE is measured every 5 epochs.

4.1.2 Network B

Figure 2 shows the learning curves for Network B. The overall trajectory is similar to Network A, with training SSE declining from approximately 5,512 to 42 and validation SSE from approximately 350 to 5.03. Network B exhibits a notably slower initial descent in the first 5 epochs (training SSE remains at approximately 5,334 at epoch 5 compared to 3,151 for Network A), suggesting that Leaky ReLU requires slightly more epochs to escape the initial loss surface. However, by epoch 80, Network B achieves a lower validation SSE than Network A at the same epoch. A sharp spike at epoch 90 (validation SSE $= 21.71$) recovers immediately, again consistent with momentum-induced overshooting.

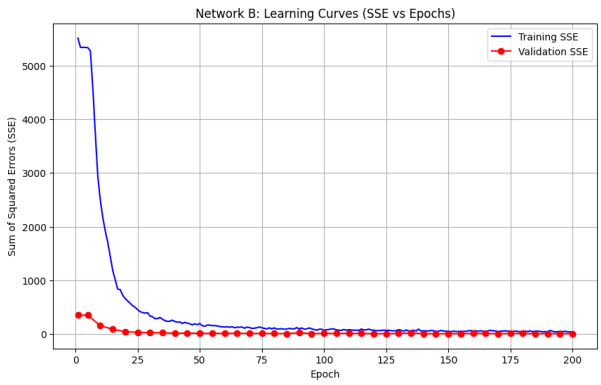

Figure 2: Network B (Leaky ReLU): Training and validation SSE versus epoch number. Validation SSE is measured every 5 epochs.

4.2 Timing Comparisons

Table 5 compares the training times for both networks over the 200-epoch run.

Table 5: Training time comparison for 200 epochs.

Network	Training Time (s)	Time per Epoch (s)
A	75.29	0.376
B	61.82	0.309

Network B trains approximately 21% faster than Network A. The Tanh forward pass involves the transcendental function $1.716 \cdot \tanh(0.667 \cdot v)$, while Leaky ReLU requires only a conditional: $\max(v,\ 0.01v)$. Similarly, during backpropagation, the Tanh derivative involves two multiplications with the output: $\frac{b}{a}(a-y)(a+y)$, whereas the Leaky ReLU derivative is a simple threshold comparison.

4.3 Confusion Matrices

Figures 3 and 4 present the confusion matrices for Networks A and B, respectively, evaluated on the 800-sample validation set.

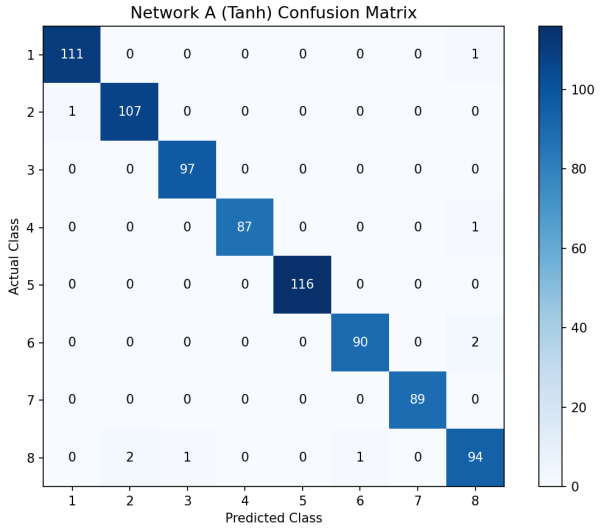

Figure 3: Network A confusion matrix on the validation set.

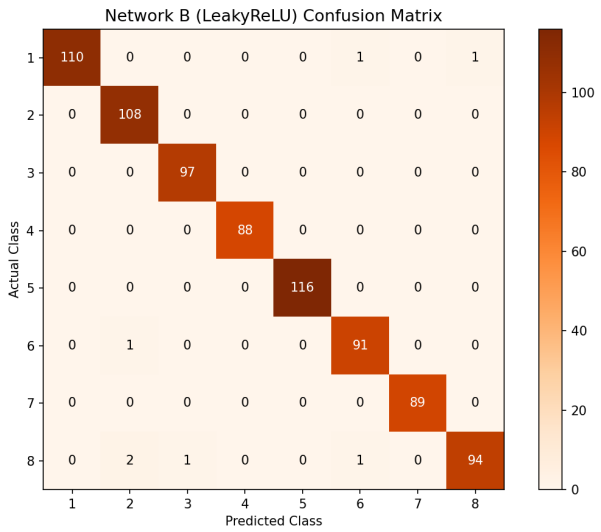

Figure 4: Network B confusion matrix on the validation set.

4.4 Performance Metrics

Table 6 presents the macro-averaged performance metrics for both networks on the validation set.

Table 6: Macro-averaged performance metrics on the validation set (800 samples).

Metric	Network A	Network B
Misclassifications	9	7
Final Validation SSE	7.1331	5.0268
Accuracy	0.9888	0.9912
Precision (macro)	0.9888	0.9913
Recall (macro)	0.9885	0.9913
F1 Score (macro)	0.9886	0.9913
MCC	0.9871	0.9900

4.5 Metrics Interpretation

Accuracy measures the proportion of correctly classified samples. Network A achieves 98.88% (791/800 correct) and Network B achieves 99.12% (793/800 correct). While high, accuracy alone can be misleading for imbalanced data; however, since our validation set is approximately balanced, it is a reliable indicator here.

Precision quantifies the fraction of predicted positives that are true positives for each class, averaged (macro) across all 8 classes. A macro precision of $\sim$0.99 for both networks indicates that when either network predicts a class, it is almost always correct.

Recall measures the fraction of actual positives correctly identified. Both networks achieve near-perfect macro recall (≥ 0.9885), confirming that the SMOTE-balanced training data enabled the networks to learn effective representations for all classes.

F1 Score, the harmonic mean of Precision and Recall, provides a single balanced measure. The high macro F1 values (0.9886 and 0.9913) confirm that neither precision nor recall is being sacrificed for the other.

Matthews Correlation Coefficient (MCC) is computed using the multiclass generalization:

$$\text{MCC} = \frac{c \cdot s - \sum_k p_k \cdot t_k}{\sqrt{(s^2 - \sum_k p_k^2)(s^2 - \sum_k t_k^2)}} \tag{2}$$

where c is the total number of correct predictions, s is the total number of samples, p_k is the number of times class k was predicted, and t_k is the number of times class k truly occurred. MCC ranges from -1 (total disagreement) to $+1$ (perfect prediction) and is considered more informative than accuracy for imbalanced datasets because it accounts for all four quadrants of the confusion matrix [4]. Both networks achieve MCC values close to 1.0 (0.9871 and 0.9900), confirming strong multiclass discrimination that is consistent across all classes, not inflated by any dominant class.

Network B edges ahead on every metric. Its lower final validation SSE (5.03 vs 7.13) suggests that Leaky ReLU's avoidance of saturation enables better gradient flow in later training epochs, where Tanh neurons in the saturated regime ($|y| \approx 1.716$) produce near-zero gradients. However, the performance difference is small: only 2 additional misclassifications separate the two networks on 800 validation samples, which is within the range of variation attributable to mini-batch shuffling. The performance gap should therefore be interpreted cautiously.

4.6 Test Set Predictions

Predictions were generated for the unlabeled test set using the saved weights of both networks. Both networks predicted only 6 of the 8 classes, Classes 2 and 3 were absent from all predictions. Predictions were skewed toward Class 8 ($\approx$32–41%) and Class 6 ($\approx$22–31%), reflecting the original class imbalance in the test set.

Table 7: Test Set Prediction Distribution

Class	Network A		Network B	
	Count	%	Count	%
Class 1	102	14.6%	88	12.6%
Class 2	—	—	—	—
Class 3	—	—	—	—
Class 4	93	13.3%	108	15.4%
Class 5	51	7.3%	60	8.6%
Class 6	160	22.8%	215	30.7%
Class 7	8	1.1%	6	0.9%
Class 8	287	40.9%	224	32.0%
Total	**701**	**100%**	**701**	**100%**

5. ANALYSIS AND DISCUSSION OF RESULTS

5.1 Effect of SMOTE on Generalization

The application of SMOTE was essential for achieving balanced per-class performance. Without it, the network would likely converge to a solution that favors Class 1 (46.6% of the original data) at the expense of Classes 3 and 7. The near-perfect recall for all classes, including the originally extreme minorities (30 and 52 instances for Classes 3 and 7), demonstrates that SMOTE successfully enriched the training distribution. On the test set, however, both networks never predict Classes 2 and 3, which is consistent with the unlabeled test set reflecting the original imbalanced distribution rather than a model deficiency: both networks achieve F1 ≥ 0.98 for these classes on the balanced validation set.

5.2 Learning Curves

Network A's learning curves show a smooth, monotonic decline in both training and validation SSE, punctuated by occasional spikes in validation SSE. These spikes are characteristic of momentum-based mini-batch SGD: with $\alpha = 0.9$, accumulated weight velocity can temporarily overshoot when a noisy batch sequence produces conflicting gradient directions. The rapid recovery after each spike confirms these are transient events rather than signs of divergence or overfitting.

Network B's learning curves reveal a notably different pattern. In the first 5 epochs, training SSE barely decreases. This delayed start occurs because Haykin's initialization heuristic, designed for bounded activations like Tanh, can produce large early activations under Leaky ReLU's unbounded regime. These large activations temporarily saturate the Logistic output layer, flattening output-layer gradients. Once weights self-rescale into an effective operating range (around epoch 10), learning proceeds rapidly. Network B ultimately converges to lower final SSE values than Network A.

5.3 Performance Comparison

Both networks achieve strong generalization, with accuracy exceeding 98.8% on the 800-sample validation set. Network B outperforms Network A on every reported metric. Its accuracy advantage of 0.24 percentage points (99.12% vs 98.88%), lower misclassification count (7 vs 9), and higher MCC (0.9900 vs 0.9871) suggest that Leaky ReLU's avoidance of gradient saturation in the hidden layers enables better weight updates throughout training.

Leaky ReLU's simple derivative, a step function returning either 1.0 or the slope parameter, is computationally cheaper than Tanh's transcendental function evaluation. This contributes to Network B's approximately 21% faster training time (61.82 s vs 75.29 s), even though the two networks share identical layer sizes and batch configurations.

5.4 Hyperparameter Tuning Analysis

The tuning experiments identified [100, 50] as the optimal hidden layer configuration. Smaller architectures like [64, 32] underfitted the 354-dimensional input, while larger architectures like [200, 100] showed only marginal improvement at significantly higher computational cost. A learning rate of 0.001 consistently outperformed 0.001, and batch size 16 offered the best balance between gradient noise and update frequency.

5.5 Test Set Prediction Analysis

The consistent absence of Classes 2 and 3 from test set predictions across both networks is most likely attributable to the composition of test set rather than a model deficiency. On the validation set, both networks achieve near-perfect F1 scores for Classes 2 and 3, confirming that the models have learned to distinguish these classes. The test set distribution, skewed toward Classes 6 and 8, mirrors the original imbalanced dataset, suggesting that it was not rebalanced with SMOTE.

5.6 Overfitting Analysis

Neither network exhibits persistent divergence between training and validation error over the 200-epoch training horizon. Both show occasional validation SSE spikes (e.g., Network A at epoch 150, Network B at epoch 90) that quickly recover. These spikes are attributed to momentum-based SGD occasionally overshooting when unfavorable batch orderings produce noisy gradient estimates. The high final validation accuracy for both networks (98.88% and 99.12%) confirms good generalization in the SMOTE-balanced dataset. No early stopping was applied, as the validation error did not show a sustained upward trend.

6. CONCLUSION

This assignment successfully implemented a two-hidden-layer MLP from scratch using NumPy, trained with mini-batch SGD and momentum via the backpropagation algorithm. SMOTE was applied to address a severe class imbalance (up to 54:1 ratio), producing a balanced training set of 12,200 samples. Systematic hyperparameter tuning identified [100, 50] hidden layer sizes with $\eta = 0.001$ and batch size 16 as optimal for both networks.

Network A (Tanh-Tanh-Logistic) achieved 98.88% accuracy and 0.9871 MCC in 75.29 seconds. Network B (LeakyReLU-LeakyReLU-Logistic) achieved 99.12% accuracy and 0.9900 MCC in 61.82 seconds, outperforming Network A on all metrics while training approximately 21% faster. Both networks generalized well, with no evidence of persistent overfitting.

The results highlight two practical advantages of Leaky ReLU over Tanh for this classification task: faster per-epoch computation due to a simpler derivative and superior final accuracy due to reduced gradient saturation in deeper hidden layers. For future work, exploring deeper architectures, dropout regularization, or adaptive optimizers (Adam, RMSProp) could further improve performance.

7. REFERENCES

[1] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, 2009.

[2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[4] D. Chicco and G. Jurman, "The advantages of the Matthews Correlation Coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 6, 2020.

[5] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.